CS 61A — Week 2

Solutions: Lab1, Discussion1, HW2, Hog, actual past-exam practice

Lab1 Q1: Return and Print. **Idea.** Record the output effect immediately, then use the returned string in the enclosing computation.

1. `welcome()` prints `Go`, then returns `'hello'`. At the prompt the returned string is displayed with quotes, so the two lines are `Go`, then `'hello'`.
2. `print(welcome(),cal())` first prints `Go`, then `Bears`, then `hello world`. The final call returns `None`, which adds no prompt display. Printing strings omits their quotes.

Lab1 Q2: WWPD: What If? **Idea.** Follow each conditional statement independently; distinguish an output line from the prompt's display of a returned value.

1. `ab(10,20)` enters the first clause, so it never tests the `elif`; then the following print still runs. Its lines are `10`, `foo`; it returns `None`.
2. `bake(0,5)` changes `cake` to 1 and prints it. The second `if` now runs and prints 5; the final return gives 5. Prompt lines: 1, 5, 5.
3. `bake(1,'yum')` skips the first test's body, prints `yum` in the second, then returns `'yum'`. Prompt lines: `yum`, `'yum'`.
4. `bake(2,3)` runs the second test's `else` and immediately returns 2, skipping the final return. The prompt displays only 2.

Lab1 Q3: Pick a Digit. **Idea.** Divide away the unwanted rightmost digits, then retain only one digit.

```
def pick_digit(n, k):
    """Return digit k from the right (positions start at zero)."""
    return (n // pow(10, k)) % 10
```

1. Dividing 3579 by 10^2 with floor division leaves 35; remainder $35\%10 = 5$ gives the requested digit.
2. At $k = 0$, division by 1 leaves the number, whose last digit is 9. If $10^k > n$, the quotient is 0, so the returned remainder is also 0.

Lab1 Q4: Middle Number. **Idea.** The sum contains all three integers once; subtract the two extremes.

```
def middle(a, b, c):
    """Return the middle of three distinct integers."""
    return a + b + c - min(a, b, c) - max(a, b, c)
```

1. For ordered values $u < v < w$, the expression becomes $u + v + w - u - w = v$.
2. For $(30, 5, 40)$ it gives $75 - 5 - 40 = 30$. The other supplied triples give $4, 5, 5$ respectively. These integer additions and subtractions are exact.

Lab1 Q5: Falling Factorial. **Idea.** Store the product already formed and move the next factor down by one each time.

```
def falling(n, k):
    product = 1
    while k > 0:
        product = product * n
        n, k = n - 1, k - 1
    return product
```

1. Initially no factors have been used, so the product is 1. An iteration multiplies by the next factor and reduces the remaining count by 1.
2. For $(6, 3)$, product/next-factor/remaining-count states are $(1, 6, 3) \to (6, 5, 2) \to (30, 4, 1) \to (120, 3, 0)$. At remaining count zero, the requested factors have all been used.
3. The $k = 0$ case returns 1 without entering the loop. The other source tests give 24 and 4.

Lab1 Q6: Divisible By k. **Idea.** Generate the multiples directly rather than testing numbers that cannot be printed.

```
def divisible_by_k(n, k):
    current, count = k, 0
    while current <= n:
        print(current)
        current, count = current + k, count + 1
    return count
```

1. Before each test, `current` is the next positive multiple of k and `count` records earlier printed multiples. Adding k visits them in order without omissions or duplicates.
2. For $(10, 2)$, print $2, 4, 6, 8, 10$ and return 5. For $(3, 1)$, print $1, 2, 3$ and return 3. For $(6, 7)$ the first condition is false: print nothing and return 0.

Lab1 Q7: Double Eights. **Idea.** Each possible adjacent pair appears as the last two digits after a suitable number of single-digit removals.

```
def double_eights(n):
    while n >= 10:
        if n % 100 == 88:
            return True
        n = n // 10
    return False
```

1. For 2882, the successive two-digit remainders are 82, then 88, so return true. For 80808080, they alternate 80 and 08 and never equal 88.
2. Removing only one digit checks overlapping pairs; once fewer than two digits remain, no unchecked pair exists, so return false. The algorithm here uses the nonnegative integer decimal-digit interpretation.

Lab1 Q8: Syllabus Quiz. **Idea.** A linked administrative form must be answered from its actual questions.

1. Use the actual linked form and official syllabus; sign-in is requested. The form's question text is unavailable here, so this packet does not contain its answer key.

Lab1 Q9 (optional): Sum Digits. **Idea.** Separate the digit just removed from the number still unprocessed.

```
def sum_digits(y):
    total = 0
    while y > 0:
        total, y = total + y % 10, y // 10
    return total
```

1. For 4224, the (total, y) states are $(0, 4224) \to (4, 422) \to (6, 42) \to (8, 4) \to (12, 0)$.
2. Each step preserves "total = sum of the removed digits". At $y = 0$, all digits are removed. Input 0 returns the initialized sum 0.

Discussion1 Q1: Race. **Idea.** Passing the hare changes the sign of the difference, but both signs are true in Python.

1. With $(x, y) = (2, 3)$, after five minutes the positions are $(10, 15)$. During the rest they become $(12, 15), (14, 15), (16, 15)$ at minutes 6,7,8. The first catch-or-pass is minute 8, but the condition remains true because $16 - 15 \neq 0$.
2. The source eventually returns minute 15 when positions are $(30, 30)$, so it gives the wrong value. Replace its loop condition by `minutes == 0 or tortoise < hare` to stop when the tortoise first reaches or passes the hare.
3. An infinite case is $(3, 4)$. In the first ten minutes, differences are $-1, -2, -3, -4, -5, -2, 1, 4, 7, 10$: no zero. At minute $10c$ the difference is $10c$. For $c \geq 1$, the next running period reduces it by at most 5, and the resting period increases it by 15; it is always positive. The source therefore never finds equality, although the first crossing is minute 7.

The worksheet says "greater distance" but its equality examples use $\geq$; the correction follows those examples. Both counterexamples pass strictly beyond, so the diagnosis does not depend on accepting equality.

Discussion1 Q2: Fizzbuzz. **Idea.** Test the most specific case first, so a multiple of 15 is not handled by the separate 3 or 5 case.

```
def fizzbuzz(n):
    i = 1
    while i <= n:
        if i % 15 == 0:
            print('fizzbuzz')
        elif i % 3 == 0:
            print('fizz')
        elif i % 5 == 0:
            print('buzz')
        else:
            print(i)
        i = i + 1
```

1. For each i, exactly one clause prints its prescribed output. Increasing i then visits every integer through n once.
2. For $n = 16$, the lines are $1, 2$, `fizz`, 4, `buzz`, `fizz`, $7, 8$, `fizz`, `buzz`, 11, `fizz`, $13, 14$, `fizzbuzz`, 16. The call returns `None`.

Discussion1 Q3: Is Prime? **Idea.** One divisor strictly between 1 and n proves compositeness; finding none proves primality only after the entire candidate range is checked.

```
def is_prime(n):
    if n < 2:
        return False
    i = 2
    while i < n:
        if n % i == 0:
            return False
        i = i + 1
    return True
```

1. Integers below 2 are not prime. Before each loop test, no integer from 2 through $i - 1$ divides n.
2. A divisor returns false immediately. Reaching $i = n$ means none from 2 through $n - 1$ divides it, so return true. For 10, divisor 2 disproves primality; for 7, all candidates fail; 1 uses the initial guard.

Discussion1 Q4: Unique Digits. **Idea.** Count possible digit values, not appearances. Each helper search should start from the original number.

```
def has_digit(n, k):
    assert k >= 0 and k < 10
    while n > 0:
        if n % 10 == k:
            return True
        n = n // 10
    return False

def unique_digits(n):
    k, count = 0, 0
    while k < 10:
        if has_digit(n, k):
            count = count + 1
        k = k + 1
    return count
```

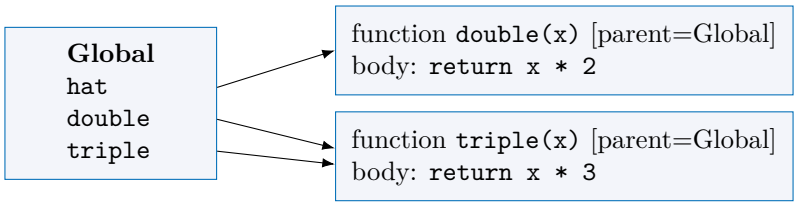
1. `has_digit` checks the current last digit and strips it after a failed match. For positive n, the loop eventually examines every decimal digit, including internal zeros.
2. `unique_digits` checks $0, 1, \ldots, 9$ exactly once, adding one for each successful search. Repeated occurrences cannot add more than one per value. Thus 101 contributes for 0 and 1, giving 2.

Discussion1 Q5: Bottles. **Idea.** The local assignment and global expression refer to different bindings with the same spelling.

1. There is one user-function call, so the diagram contains the global frame plus one local call frame. Count calls, not definitions or return statements.
2. That frame has `around=99`, local `bottles=98`. It finds global `take=1` and returns 1. The global expression computes $99 - 1 = 98$, binding `remaining`.
3. The local assignment has no effect on global `bottles`. The *later top-level* `bottles=remaining` finally rebinds the global name to 98.

Discussion1 Q6: Double Trouble. **Idea.** A second name retains the function value that was present when the assignment ran.

1. Initially global `double` refers to the function returning `x*2`, and `triple` to the function returning `x*3`.
2. `hat=double` binds `hat` to the first function. `double=triple` then binds `double` to the second, leaving `hat` unchanged. Both functions have parent Global; no call frames exist yet.
3. Therefore `hat(4)` returns 8, and `double(4)` returns 12. The function now named `double` still has intrinsic name `triple`.



Before the two additional check calls, only the Global frame exists. `hat` retains the original double function; `double` and `triple` refer to the same triple function.

Discussion1 Q7 (optional): Repeating. **Idea.** Every block must equal the saved last block and have the specified width. A numeric remainder alone does not remember leading zeros.

```
1  def repeating(t, n):
2      if pow(10, t - 1) > n:
3          return False
4      end = n % pow(10, t)
5      if end < pow(10, t - 1):  # Added guard: block must have t digits.
6          return False
7      while n:
8          if n % pow(10, t) != end:
9              return False
10         n = n // pow(10, t)
11     return True
```

1. The source's first guard ensures the entire number has at least t digits. The added guard ensures the saved block itself has t digits; otherwise no t-digit positive integer could be the repeated block.
2. Each successful comparison removes one full block. If fewer than t digits remain, their value is less than 10^{t-1} and cannot equal the saved block. Thus success at $n = 0$ means exactly a whole number of identical blocks were removed.
3. For $6161, t = 2$, compare 61, remove it to get 61, compare again, then reach 0: true. At $t = 3$, the remainder is 161 but the remaining 6 differs: false. For $101, t = 2$, the added guard rejects the final block's numeric value 1.

The guard is an explicit correction to the source template, rather than an unreported change to its mathematical contract.

§1 Homework 2

The supplied helper contracts are simple arithmetic functions:

```
from operator import add, mul

def square(x):
    return x * x

def identity(x):
    return x

def triple(x):
    return 3 * x

def increment(x):
    return x + 1
```

HW2 Q1: Product. **Idea.** Use the shared summation argument with multiplication and initial product 1.

```
def product(n, term):
    """Return the product of term(1) through term(n)."""
    result, k = 1, 1
    while k <= n:
        result, k = result * term(k), k + 1
    return result
```

1. Before each test at k, `result` is the product of terms 1 through $k-1$. Multiplying by the next term preserves the property after increasing k.
2. At $k = n+1$, return the full product. The six supplied examples give $6, 120, 36, 14400, 24, 162$ in their stated order.

HW2 Q2: Accumulate. **Idea.** Parameterize both the combination and its initial value; the two wrappers then choose their respective identities.

```
def accumulate(fuse, start, n, term):
    """Combine start with the first n terms using fuse."""
    result, k = start, 1
    while k <= n:
        result, k = fuse(result, term(k)), k + 1
    return result

def summation_using_accumulate(n, term):
    """Sum the first n terms using accumulate."""
    return accumulate(add, 0, n, term)

def product_using_accumulate(n, term):
    """Multiply the first n terms using accumulate."""
    return accumulate(mul, 1, n, term)
```

1. After j iterations, the result is the left-to-right combination of `start` with the first j terms. One more call to `fuse` extends it by the next term. The assumed associativity and commutativity justify the source's alternate nesting order.
2. At $n = 0$, the loop does nothing and returns `start`. For start 11 and the first three squares, results are $11, 12, 16, 25$.
3. Addition's identity is 0, multiplication's 1, so the wrappers need exactly one return statement each. Their supplied outputs are $55, 45$ and $576, 524880$. The custom `lambda x,y:x+y+1` combines start 2 and squares $1, 4, 9$ as $2 \to 4 \to 9 \to 19$.

HW2 Q3: Make Repeater. **Idea.** Retain the function and repetition count in the defining frame, but start a fresh current value and counter on each call.

```
def make_repeater(f, n):
    def repeated(x):
        count = 0
        while count < n:
            x, count = f(x), count + 1
        return x
    return repeated
```

1. After `count` iterations, the current value is `f` applied `count` times to the original input. One update and increment preserve this statement.
2. Reaching count $= n$ returns the requested iteration. Three increments at 5 give 8; five triples at 1 give 243; two/three squares at 5 give 625/390625.

HW2 Q4: Composite Identity Function. **Idea.** Capture both function values and postpone their comparison until an input is supplied.

```
def composite_identity(f, g):
    def equal_at(x):
        return f(g(x)) == g(f(x))
    return equal_at
```

1. Each call to the returned function evaluates both compositions at its own `x`; equality returns a Boolean directly.
2. For square and increment at 0, both values are 1: true. At 4, the two values are 25 and 17: false. No single-input result asserts that the functions commute everywhere.

HW2 Q5: Church basic numerals and conversion (just for fun). **Idea.** Make the numeral's returned function apply its supplied operation the required number of times.

```
def zero(f):
    return lambda x: x

def successor(n):
    return lambda f: lambda x: f(n(f)(x))

def one(f):
    return lambda x: f(x)

def two(f):
    return lambda x: f(f(x))

three = successor(two)

def church_to_int(n):
    return n(lambda x: x + 1)(0)
```

1. Zero returns its input unchanged; one applies `f` once; two twice. The latter two do not call `successor`, as required.
2. Successor adds one application outside the existing n applications. Conversion uses increment at initial 0, so zero, one, two and three convert to $0, 1, 2, 3$.

HW2 Q5: Church arithmetic (continued). **Idea.** Combine application counts while leaving the supplied function arbitrary.

```
def add_church(m, n):
    return lambda f: lambda x: m(f)(n(f)(x))

def mul_church(m, n):
    return lambda f: m(n(f))

def pow_church(m, n):
    return n(m)
```

1. Addition first performs n applications to `x`, then m more. The result converts to $m + n$; source example $2 + 3 = 5$.
2. Multiplication repeats the n-application function m times, giving mn applications. Source examples give $2 \cdot 3 = 6$ and $3 \cdot 4 = 12$.
3. Exponentiation applies M to functions n times. At M = two, each application doubles the repetition count: $1 \to 2 \to 4 \to 8$. Hence `pow_church(two,three)` converts to 8; reversing them gives 9. At exponent zero, `zero(m)` returns the identity on functions, representing numeral one; the implementation uses Python's convention $0^0 = 1$.

§2 Hog project

Use the supplied `dice.py` in the assignment folder; these imports and the goal are provided setup, not new implementations.

```
from dice import six_sided, make_test_dice
GOAL = 100
```

Hog P0: Dice. **Idea.** A test die's retained position advances on every call.

1. The five results for the $4, 1, 2$ cycle are $4, 1, 2, 4, 1$. Returning different outcomes on successive zero-argument calls makes this die non-pure.

Hog P1: Roll Dice. **Idea.** Remember whether any one occurred without ending the loop that consumes the dice.

```
def roll_dice(num_rolls, dice=six_sided):
    assert type(num_rolls) == int, 'num_rolls must be an integer.'
    assert num_rolls > 0, 'Must roll at least once.'
    total, rolled_one, count = 0, False, 0
    while count < num_rolls:
        outcome = dice()
        total = total + outcome
        if outcome == 1:
            rolled_one = True
        count = count + 1
    if rolled_one:
        return 1
    return total
```

1. Before each test, `count` is the number of outcomes already consumed, `total` their sum, and `rolled_one` records whether any was 1.
2. The body consumes exactly one more outcome and preserves all three meanings. At count = `num_rolls`, choose the required score. With cyclic outcomes $4, 1, 2, 6$, three rolls return 1 and leave 6 as the next outcome.

Hog P2: Boar Brawl. **Idea.** Use place-value arithmetic, retaining only the specified digits even for scores above 99.

```
def boar_brawl(player_score, opponent_score):
    own_ones = player_score % 10
    other_tens = (opponent_score // 10) % 10
    return max(3 * abs(other_tens - own_ones), 1)
```

1. Own score 21 has ones 1; opponent 46 has tens 4, giving $3|4-1| = 9$. With $(45, 52)$ the difference is zero, so the award is 1.
2. For opponent 146, division by 10 gives 14 and remainder by 10 gives tens digit 4. Omitting the final remainder would wrongly use 14.

Hog P3: Take Turn. **Idea.** Reuse the functions whose contracts already implement the two possible turn rules.

```
def take_turn(num_rolls, player_score, opponent_score, dice=six_sided):
    assert type(num_rolls) == int, 'num_rolls must be an integer.'
    assert num_rolls >= 0, 'Cannot roll a negative number of dice in take_turn.'
    assert num_rolls <= 10, 'Cannot roll more than 10 dice.'
    if num_rolls == 0:
        return boar_brawl(player_score, opponent_score)
    return roll_dice(num_rolls, dice)
```

1. Zero rolls return the Boar Brawl award without calling the die. Positive rolls return `roll_dice`'s result using the supplied die.
2. Thus two outcomes $4, 5$ give 9; three outcomes $4, 6, 1$ give 1; a zero-roll turn at scores $(12, 41)$ gives $3|4-2| = 6$.

Provided Hog helpers. The first adds turn points without Sus Fuss. The primality helper's stated domain is positive integers; unlike the general Discussion1 solution, it is not a test for zero/negative integers. The two supplied strategies are used in later project tests.

```
def simple_update(num_rolls, player_score, opponent_score, dice=six_sided):
    return player_score + take_turn(num_rolls, player_score, opponent_score, dice)

def is_prime(n):
    """Provided Hog helper; domain is positive integers."""
    if n == 1:
        return False
    k = 2
    while k < n:
        if n % k == 0:
            return False
        k += 1
    return True

def always_roll_5(score, opponent_score):
    return 5

def catch_up(score, opponent_score):
    if score < opponent_score:
        return 6
    else:
        return 5
```

Hog P4: Sus Fuss. **Idea.** First compute the new total, then decide whether that total's divisor count triggers replacement.

```
def num_factors(n):
    count, k = 0, 1
    while k <= n:
        if n % k == 0:
            count = count + 1
        k = k + 1
    return count

def sus_points(score):
    factors = num_factors(score)
    if factors == 3 or factors == 4:
        candidate = score + 1
        while not is_prime(candidate):
            candidate = candidate + 1
        return candidate
    return score

def sus_update(num_rolls, player_score, opponent_score, dice=six_sided):
    return sus_points(simple_update(num_rolls, player_score, opponent_score, dice))
```

1. The inclusive divisor loop counts every positive divisor once. For $1, 2, 3, 8, 9$, the counts are $1, 2, 2, 4, 3$.
2. Only counts 3 or 4 start the next-prime search. Testing candidates in increasing order gives the nearest greater prime; the search eventually succeeds by the existence of arbitrarily large primes (cited).
3. At starting score 1 with a 5-point turn, `simple_update` gives 6; `sus_update` gives 7 because 6 has four factors. Testing the award 5 or old score 1 would miss that change.

Hog P5: Play. **Idea.** Let the turn variable choose which score is current, while keeping the final score pair in fixed player order.

```
def play(strategy0, strategy1, update, score0=0, score1=0,
         dice=six_sided, goal=GOAL):
    who = 0
    while score0 < goal and score1 < goal:
        if who == 0:
            rolls = strategy0(score0, score1)
            score0 = update(rolls, score0, score1, dice)
        else:
            rolls = strategy1(score1, score0)
            score1 = update(rolls, score1, score0, dice)
        who = 1 - who
    return score0, score1
```

1. Before every test, both scores are the totals after previous turns and `who` identifies the next player. Call its strategy once and replace its score by the update result once.
2. For Player1, `score1` is passed first and `score0` second; the storage order of the final pair does not change. Switching `who` then preserves the statement for the next turn.
3. When either total reaches the goal, the loop stops before calling another strategy. Already-finished starting scores also skip the loop. Under the supplied rules every turn increases a score, proving termination.

Hog P6: Always Roll. **Idea.** Capture the chosen roll count and accept the two score arguments required by the strategy interface.

```
def always_roll(n):
    assert n >= 0 and n <= 10
    def strategy(score, opponent_score):
        return n
    return strategy
```

1. The outer call validates and binds `n`. Its inner function has that call frame as parent, so every later call finds the same retained `n`.
2. Thus `always_roll(3)(10,20)` is 3, and `always_roll(0)(99,99)` is 0.

Hog P7: Is Always Roll. **Idea.** A deterministic strategy's legal score domain is finite; enumerate the full Cartesian set of pairs, not just one diagonal.

```
def is_always_roll(strategy, goal=GOAL):
    expected = strategy(0, 0)
    score = 0
    while score < goal:
        opponent = 0
        while opponent < goal:
            if strategy(score, opponent) != expected:
                return False
            opponent = opponent + 1
        score = score + 1
    return True
```

1. The outer loop chooses every own score below the positive goal, and the inner loop every opponent score below it. A mismatch disproves constancy.
2. If both loops finish, every pair has the reference value. `always_roll_5` and `always_roll(3)` pass; `catch_up` fails because equal and behind positions differ. The argument assumes a deterministic strategy.

Hog P8: Make Averaged. **Idea.** Retain the sampled function and sample count, then forward each later call's arguments unchanged.

```
def make_averaged(func_to_average, iterations=1000):
    def averaged(*args):
        total, count = 0, 0
        while count < iterations:
            total = total + func_to_average(*args)
            count = count + 1
        return total / iterations
    return averaged
```

1. For positive integer `iterations`, after `count` passes, `total` is the sum of exactly that many results of calls on the supplied arguments.
2. After the last pass, divide by the number of samples. For die outcomes $4, 2, 5, 1$ and 40 one-roll samples, each outcome appears ten times and the result is 3.0. A second call starts a new total, while using the same captured function.

Hog P9: Max Scoring Num Rolls. **Idea.** Compare estimates in increasing roll-count order and retain the earlier choice on a tie.

```
def max_scoring_num_rolls(dice=six_sided, iterations=1000):
    average = make_averaged(roll_dice, iterations)
    rolls, best_rolls, best_score = 1, 1, 0
    while rolls <= 10:
        score = average(rolls, dice)
        if score > best_score:
            best_rolls, best_score = rolls, score
        rolls = rolls + 1
    return best_rolls
```

1. Every die outcome is positive, so the first sampled mean is greater than initialized best score 0. Each later strict improvement replaces both the score and its count.
2. Equal means leave the earlier, smaller count intact. Constant dice 3 or 2 return 10 rolls; constant die 1 returns 1. Alternating $1, 2$ also returns 1: one roll averages 1.5, whereas any two or more rolls include 1 and score 1.

Hog P10: Boar Strategy. **Idea.** The threshold concerns the zero-roll award itself; this strategy deliberately ignores Sus Fuss.

```
def boar_strategy(score, opponent_score, threshold=11, num_rolls=6):
    if boar_brawl(score, opponent_score) >= threshold:
        return 0
    return num_rolls
```

1. At scores $(40, 51)$, the award is $3|5 - 0| = 15$. Return zero at thresholds 7 or 15.
2. At threshold 16, return the fallback roll count instead; the source's explicit fallback 7 is returned unchanged.

Hog P11: Sus Strategy. **Idea.** Apply the full zero-roll update and subtract the starting total to recover the gain.

```
def sus_strategy(score, opponent_score, threshold=11, num_rolls=6):
    gain = sus_update(0, score, opponent_score) - score
    if gain >= threshold:
        return 0
    return num_rolls
```

1. At scores $(53, 60)$, the total after Boar Brawl is 62, with four factors $1, 2, 31, 62$. Sus Fuss changes it to 67.
2. The gain is $67 - 53 = 14$, so threshold 14 chooses zero; threshold 15 chooses the supplied fallback. Comparing 67 directly to the threshold would be wrong.

Hog P12 (optional): One Final Strategy. **Idea.** Take a zero-roll win whenever it is certain, otherwise retain the justified Sus threshold strategy.

```
def final_strategy(score, opponent_score):
    """Take a certain win with zero dice; otherwise use the Sus threshold."""
    if sus_update(0, score, opponent_score) >= GOAL:
        return 0
    return sus_strategy(score, opponent_score, threshold=11, num_rolls=6)
```

1. If the zero-roll updated total is at least 100, that turn ends the game in our favor, so choose it without risking dice.
2. Otherwise the existing Sus rule returns either 0 or 6, always a legal count. This is a concrete valid strategy, not a proof of optimality; the optional task admits many answers.

§3 Actual past-exam practice

Fall2022 Midterm1 Q1(c), Lecture5 reproduction. **Idea.** Every print returns `None`, so the `or` must evaluate its right side.

1. `print(s,s)` displays `Knock Knock` and returns `None`. The right operand prints `Who's There?` and also returns `None`.
2. The next enclosing print receives that `None`, displays `None`, and returns `None`. The outer print receives `None` and `'Who?'`, displaying `None Who?`.

The four lines are therefore:

```
Knock Knock
Who's There?
None
None Who?
```

Fall2022 Midterm1 Q3(a), Lecture5 reproduction. **Idea.** Sum proper factors using a loop that excludes the number itself.

```
def classify(n):
    total, k = 0, 1
    while k < n:
        if n % k == 0:
            total = total + k
        k = k + 1
    if total == n:
        return 'perfect'
    elif total < n:
        return 'deficient'
    else:
        return 'abundant'
```

1. Before each test, `total` is the sum of divisors below k. On termination it includes every proper factor but not n itself.
2. For 6, sum $1+2+3=6$: perfect. For 24, sum $1+2+3+4+6+8+12=36>24$: abundant. For prime 23, sum $1<23$: deficient.

Fall2026 Midterm1 Q3: Long Odds. **Idea.** Count each maximal group once, at the instant its size first reaches the threshold.

Blank(a) is `group_size == k`; blank(b) is `n % 2 != (n // 10) % 2`. These expressions obey all restrictions. The full supplied loop with those fills is:

```
def num_long_groups(n, k):
    assert n > 0 and k > 0
    group_size, count = 0, 0
    while n:
        group_size += 1
        if group_size == k:
            count += 1
        if n % 2 != (n // 10) % 2:
            group_size = 0
        n = n // 10
    return count
```

1. Scanning right to left, the current integer has the parity of its last digit, since 10 is even. `n//10` has the parity of the digit immediately to its left. Different parities mark a group boundary.
2. Increase the group size before checking the threshold. Equality $= k$ occurs exactly once in a qualifying group; testing $\geq k$ would count its extra digits again.
3. Reset on a boundary, then strip one digit. At the final digit, whether the size resets is irrelevant because the number becomes zero. For 15732429999, sizes $4, 3, 4$ give counts $3, 2, 0, 3$ at the four source thresholds.

Fall2026 Midterm1 Q4.1–Q4.2: Space Between. **Idea.** Visit every adjacent index pair, then reuse that same function through an odd-index transformation.

```
def gap(n, term):
    k, largest = 1, 0
    while k < n:
        largest = max(largest, abs(term(k + 1) - term(k)))
        k = k + 1
    return largest

def odd_gap(n, term):
    return gap((n + 1) // 2, lambda k: term(2 * k - 1))
```

1. The correct condition is `k < n`, visiting pairs $(1, 2)$ through $(n-1, n)$. Update `largest` by taking the maximum of its old value and the current absolute difference.
2. There are $(n+1)//2$ odd indices at most n; the j th is $2j-1$. The lambda translates `gap`'s ordinary indices into these original indices.
3. For `gap(5,square)`, terms $1, 4, 9, 16, 25$ have differences $3, 5, 7, 9$, giving 9. `gap(6,lambda x:x%4)` has terms $1, 2, 3, 0, 1, 2$, giving 3; `gap(4,lambda x:-x)` gives 1.
4. `odd_gap(5,square)` compares $1, 9, 25$, with differences $8, 16$, so returns 16. `odd_gap(8,lambda x:x%5)` compares $1, 3, 0, 2$, with differences $2, 3, 2$, so returns 3. Both implementations obey the stated syntax restrictions.